

Signal and Image Processing MOD.1 - E02

Bachelor of Science in Artificial Intelligence (A.Y. 2023/2024)

University of Pavia / University of Milano - Bicocca

Exercises by dr. Marco Buzzelli

Outline

- **Local operators**, also known as **filters**, analyze the neighborhood of each pixel to modify that pixel
- We will see different types of local operators:
 - * **Linear** filtering
 - * * Blurring
 - * * Edge detection
 - * **Non-linear** filtering
 - * * Order/rank filter
- We will compare specialized and generalized functions for image filtering
- Filters can also be used to **increase the perceived detail of an image (sharpness)**
- We will perform mathematical operations on images to implement image **sharpening**

Table of Contents

Local processing (filtering).....	1
Image blurring.....	2
Local average.....	2
Weighted gaussian average with imfilter.....	3
Weighted gaussian average with imgaussfilt.....	5
Edge detection.....	6
Order filter.....	8
Median.....	8
Minimum and maximum filter.....	10
Image sharpening.....	12
Unsharp masking.....	12
Assignment (optional).....	13

Local processing (filtering)

```
% Import image, convert to grayscale floating point, and visualize
im = imread('boat_small.jpg');
im = rgb2gray(im2double(im));
figure, imshow(im), title('Grayscale image')
```

Grayscale image



Image blurring

Local average



```
% Create square window of elements summing up to 1
W = 11;
h_average = ones(W)/(W*W);
disp(h_average);
```

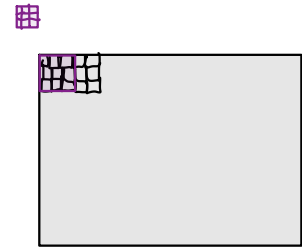
```
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.0083    0.
```

```
disp(sum(h_average(:)))
```

```
1.0000
```

```
% Apply average blur
im_blurred = imfilter(im, h_average, 'replicate');
figure, imshow(im_blurred), title('Blurred image (imfilter with average filter)')
```

Blurred image (imfilter with average filter)



<https://it.mathworks.com/help/matlab/ref/ones.html>

<https://it.mathworks.com/help/images/ref/imfilter.html>

Note:

- Our first attempt at image blurring is implemented with a 11×11 filter of equal weights
- This will be placed as a sliding window over the image
- * For every placement of the sliding window over the image, the underlying image pixels are weighted by the filter
- * The results are summed, and use to "replace" the central pixel in the new resulting image
- The filter values add up to 1, so that it effectively implements an average of image pixel values
- We add padding (a frame of extra pixels) around the image before filtering, so that the sliding window can be also located with a center on pixels around the image edge
- * We set padding mode to 'replicate', which reuses pixel values on the image edge to create this frame of extra pixels
- It is useful to create an odd-sized filter, so that there is always a "center" pixel in the filter that corresponds to one specific pixel of the image window

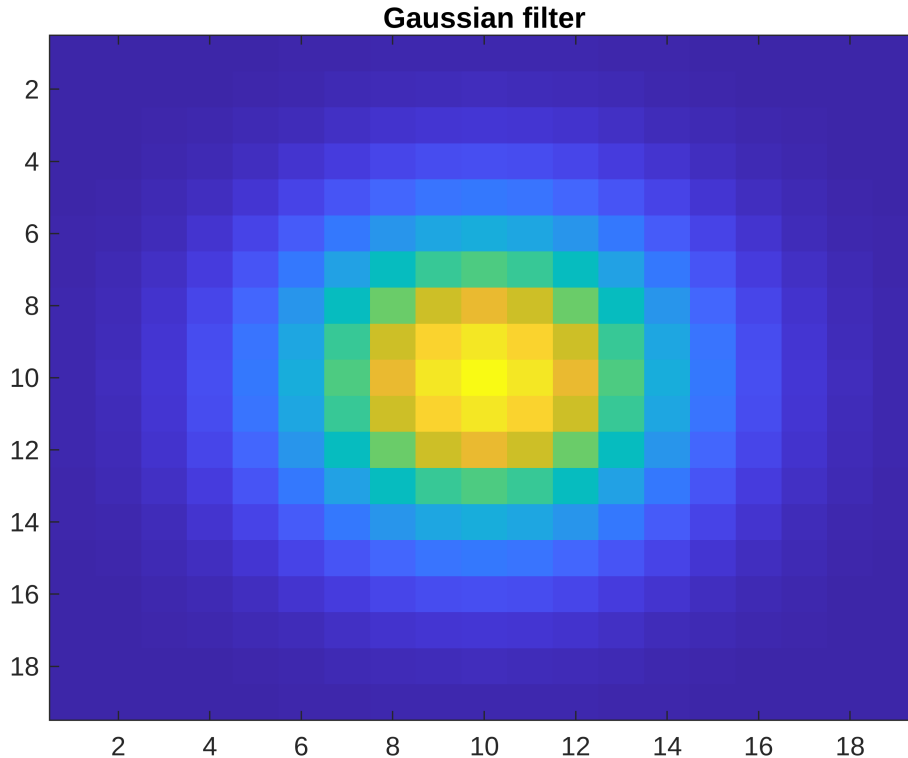
Weighted gaussian average with imfilter

```
sigma = 3; % Standard deviation for a 2-dimensional Gaussian
B = 3; % Break-off value related to sigma
W = (floor(B*sigma+0.5)*2)+1; % Window size
disp(W);
```

19

```
% Create and visualize the 2D Gaussian filter
```

```
h_gaussian = fspecial('gaussian', W, sigma);
figure, imagesc(h_gaussian), title('Gaussian filter');
```



```
disp(sum(h_gaussian(:)))
```

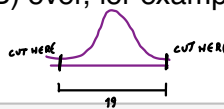
1.0000

<https://it.mathworks.com/help/images/ref/fspecial.html>

<https://it.mathworks.com/help/matlab/ref/imagesc.html>

Note:

- We are now using a weighted average for blurring
- * Specifically, we use a 2-dimensional Gaussian function
- * This assigns more weights to the center pixel in the sliding window, and gradually less weights to pixels outside
- A Gaussian function can be expressed in terms of its standard deviation (sigma)
- * We need to locate our Gaussian filter into a properly sized window
- * Theoretically, the domain of a Gaussian function extends from -infinity to +infinity
- * In practice, we can set a breakoff value (B) over, for example, 3 times sigma:
- * Window size = $\lfloor (B \cdot \sigma + 0.5) \cdot 2 \rfloor + 1 = 19$



```
% Apply gaussian blur
im_blurred2 = imfilter(im, h_gaussian, 'replicate');
figure, imshow(im_blurred2), title('Blurred image (imfilter with Gaussian filter)')
```

Blurred image (imfilter with Gaussian filter)



Note:

- Using a weighted average also results in a blurred image
- Notice the difference, for example, on the thin white tower on the right:
 - * Average filter creates a blurred version of the tower, but this appears sharply within the sky
 - * Gaussian filter creates a blurred version of the tower that gradually emerges from the sky

Weighted gaussian average with imgaussfilt

```
% Apply gaussian blur with specialized function
im_blurred3 = imgaussfilt(im, sigma);
figure, imshow(im_blurred3), title('Blurred image (imgaussfilt)')
```

Blurred image (imgaussfilt)



<https://it.mathworks.com/help/images/ref/imgaussfilt.html>

Note:

- Gaussian blur can also be directly implemented with a **specialized function**
- * This is an optimized version that should be preferred over `imfilter`, if we specifically want Gaussian blur
- * It is implemented aiming at high efficiency and reduced loss of information

Edge detection

```
% 3x3 filter for horizontal edges
h_edge = [1, 2, 1; 0, 0, 0; -1, -2, -1];
disp(h_edge)
```

```
1     2     1
0     0     0
-1    -2    -1
```

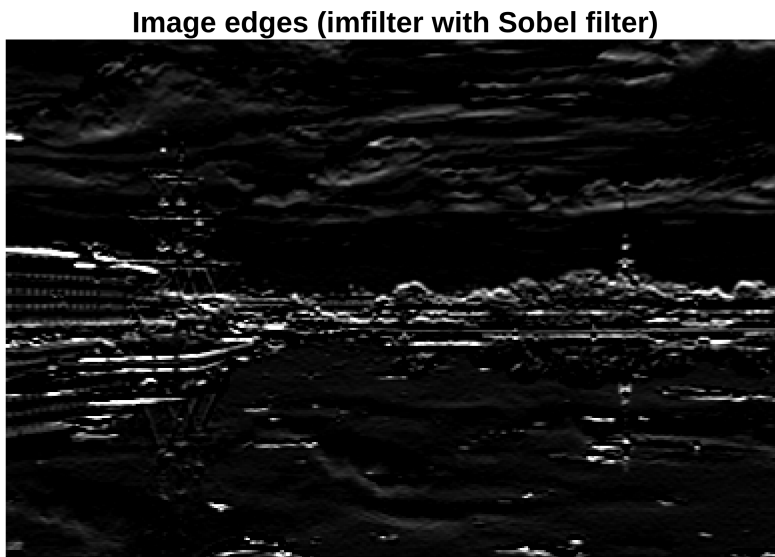
```
disp(sum(h_edge(:)))
```

```
0
```

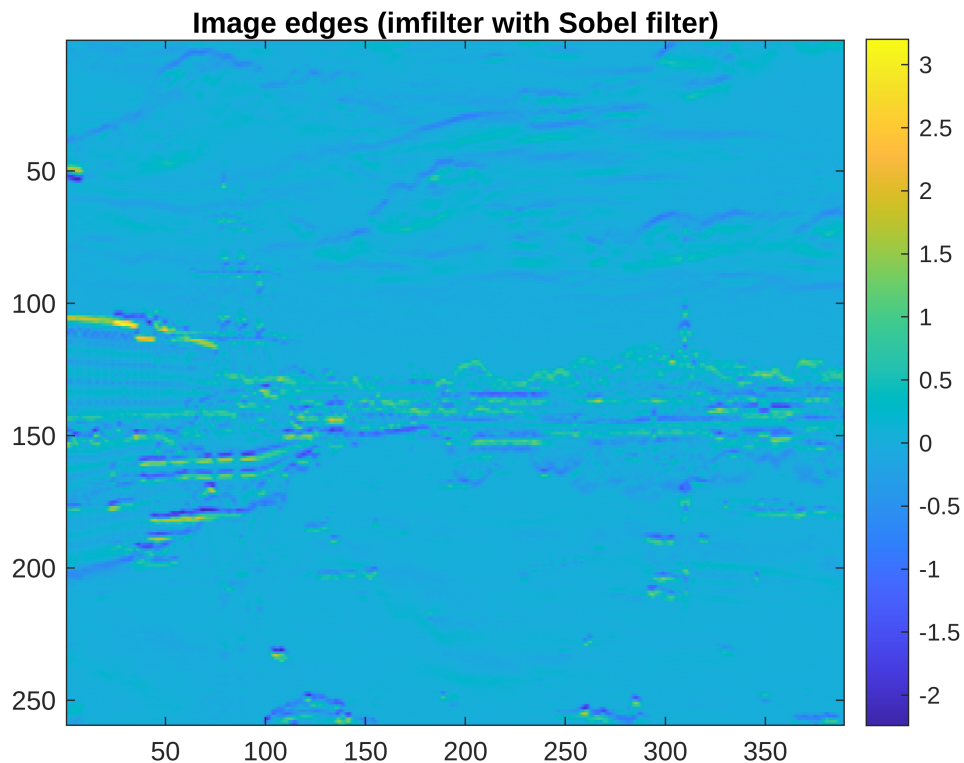
Note:

- We create a **3-by-3 filter that detects horizontal edges** using the smoothing effect by approximating a vertical gradient.
- * If vertical edges are required, the filter can be simply transposed with the operator (`'`) as: `h_edge'`
- With this type of filter, the **values of the filter do not add up to 1, but to 0.**
- * The effect of this is that we will not get as output a natural-looking image

```
% Apply edge detection
im_edges = imfilter(im, h_edge, 'replicate');
figure, imshow(im_edges), title('Image edges (imfilter with Sobel filter)')
```



```
% Visualize both positive and negative values of the computed edge map
figure, imagesc(im_edges), title('Image edges (imfilter with Sobel filter)'), colorbar
```



Note:

- "Traditional" image visualization can be used to visualize the edge map, but it is not appropriate
- * The edge map will, in fact, contain both positive and negative values. But `imshow` will clip everything below zero to black
- We use instead the `imagesc` function:
- * It scales the visualization to always show the whole minimum-maximum range of the input matrix
- * It uses a `colormap for visualization` (can be customized)
- In the context of convolutional neural networks, a linear filter is used on so-called convolutional layers
- * One key difference is that in convolutional neural networks the content of the filter is learned automatically

```
% Create edge detection filter using fspecial
h_edge = fspecial('sobel');
disp(h_edge)
```

```
1     2     1
0     0     0
-1    -2    -1
```

https://en.wikipedia.org/wiki/Sobel_operator

Note:

- The same edge-detection filter can also be generated with the `fspecial` function that we used to generate the Gaussian filter
- In this case, the filter takes the name of `Sobel filter`, by the name of its inventor

Order filter

Median

```
% Apply order-statistic filter
W = 7; % Window size
order = ceil((W*W)/2); % 50th percentile within our window
disp(order);
```

```
25
```

```
im_rank = ordfilt2(im, order, ones(W), 'symmetric');
figure, imshow(im_rank), title('Median-filtered image (ordfilt2)')
```

Median-filtered image (ordfilt2)



<https://it.mathworks.com/help/matlab/ref/ceil.html>

<https://it.mathworks.com/help/images/ref/ordfilt2.html>

Note:

- Linear filters (blur, edge, etc.) are named like this because they compute the output as a linear combination (weighted average) of the input
- We can also use non linear filters:
 - * There is still a sliding window component
 - * But the operation performed on each position is not a linear combination
- An order-statistic filter, a.k.a. a rank filter, is a non linear filter that replaces each pixel in a sliding window with the N-th ordered element (quantile)
- * In this case, we have created a 7x7 sliding window, composed therefore of 49 elements ↑
"half" of 49
- * For every position of the sliding window, we sort the 49 image pixel values, and take the 25th element
- * * Note that this corresponds to taking the median value for each sliding window position
- The result is an image that is blurred on areas that were *relatively* smooth to begin with, but that preserves strong edges
- * This idea is at the basis of beautification filters, to remove skin imperfections without blurring the whole image

```
% Apply specialized median filter
im_median = medfilt2(im, [W W]);
figure, imshow(im_median), title('Median-filtered image (medfilt2)')
```

Median-filtered image (medfilt2)



<https://it.mathworks.com/help/images/ref/medfilt2.html>

Note:

- If we are specifically interested in the 50th order percentile, we can use the specialized `medfilt2` function

Minimum and maximum filter

```
% Apply minimum filter Minimum
im_min = ordfilt2(im, 1, ones(W), 'symmetric');
figure, imshow(im_min), title('Minimum-filtered image (ordfilt2)')
```

Minimum-filtered image (ordfilt2)



```
% Apply maximum filter maximum  
im_max = ordfilt2(im, W*W, ones(7), 'symmetric');  
figure, imshow(im_max), title('Maximum-filtered image (ordfilt2)')
```

Maximum-filtered image (ordfilt2)



Note:

- By changing the order (quantile) to select, we can effectively implement a *minimum* filter, and a *maximum* filter
- In this case we have used a large filter for ease of visualization
- However, they are typically implemented with a much smaller window to perform:

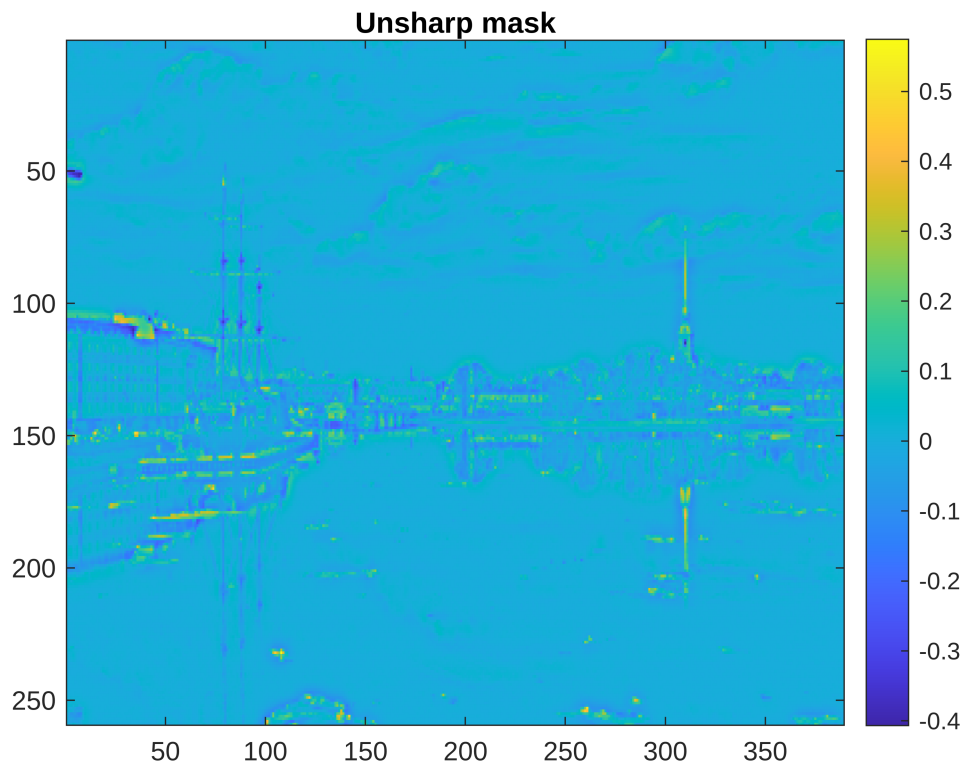
- * Salt noise removal (minimum)
- * Pepper noise removal (maximum)
- * Salt & pepper noise removal (median)
- The maximum filter also corresponds to the "max-pooling" operation seen in convolutional neural networks

Image sharpening

https://en.wikipedia.org/wiki/Unsharp_masking

Unsharp masking

```
% Sharpening based on image-level mathematic manipulation
% Create unsharp mask
sigma = 3;
im_original = im;
im_blurred = imgaussfilt(im, sigma);
unsharp_mask = im_original - im_blurred;
figure, imagesc(unsharp_mask), colorbar, title('Unsharp mask');
```



Note:

- We have created an *unsharp mask*
- * Its name derives from the fact that the technique uses a blurred, or "unsharp", version of the image

- By subtracting a blurred version from the original image, we obtain an image with only the details (or high frequencies)
- * The overall values (low frequencies) were present in both the blurred and original images, so they have been cancelled out by the subtraction

```
% Apply unsharp mask
im_sharpened = im_original + unsharp_mask;
figure, imshow(im_sharpened), title('Sharpened image (unsharp masking)');
```

Sharpened image (unsharp masking)



Note:

- We have increased the *perceived* level of detail in the image, by enhancing the change between dark and bright values

Assignment (optional)

Implement the same sharpening operation with a single, appropriate, image filter.

1. Import the image `boat_small.jpg`.
2. Create `h_gaussian`: a Gaussian filter that will be the starting point for a sharpening filter.
3. Create `h_identity`: a second filter of the same size, with all zeros except a "1" in the very center.
4. Create the final sharpen filter by combination of Gaussian and identity: $h_sharpen = h_identity + (h_identity - h_gaussian)$.
5. Visualize the sharpen filter itself, both as an image and by printing its numerical values.
6. Apply the filter to the boat image, and visualize the result.